

1 Self-supervised learning

Трансформер — очень мощная модель, требующая большого количества данных для обучения (сколько конкретно мы разберем чуть позже). Однако для многих задач разметка данных является очень время затратным и дорогостоящим процессом. Например, для решения задачи машинного перевода с одного языка на другой, понадобится набор пар предложений на двух языках, а для задачи генерации изображения по тексту — соответственно пары “изображение - описание”.

Вспомним, что веса нашей модели инициализируются случайным образом. Большой объем данных нужен нам, чтобы добиться “правильных” параметров для задачи и не переобучиться под нее. Что же делать, если данных нет? Тут на помощь приходит концепция self-supervised learning (SSL, обучение с самоконтролем).

Что такое **self-supervised learning (SSL)**? Это метод обучения нейронных сетей, при котором модель генерирует метки самостоятельно, используя исходные данные. В SSL обычно выделяют две ключевые задачи. Первая — предлоговая задача (pretext task), где из исходных данных создаётся разметка, на которой модель обучается. В процессе она формирует представления данных, основанные на генерализации извлечённой информации. Вторая — прикладная задача (downstream task), для которой модель применяется после предобучения. Для её решения часто требуется дообучение модели на небольшом объёме размеченных данных. Такой подход позволяет эффективно использовать неразмеченные данные и минимизировать ресурсы, необходимые для разметки данных.

В этом разделе мы лишь слегка коснемся методов self-supervised learning, связанных с обработкой текста.

1.1 BERT

Рассмотрим модель BERT, которая является трансформером-кодировщиком.

1.1.1 Архитектура BERT

В статье “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding” от Google AI Language, опубликованной в 2019ом году, была представлена архитектура BERT. Модель спроектировали так, чтобы использовать ее в качестве предобученного кодировщика для получения векторных представлений неразмеченного текста с учетом левого и правого контекста. За основу модели BERT берется Transformer Encoder с добавленным [CLS] токеном, который мы уже разбирали ранее.

Зачем вообще учитывать оба направления контекста? Двунаправленное внимание позволяет модели учитывать как предшествующие, так и последующие токены, обеспечивая более полное понимание контекста. Это особенно важно в задачах, где значение слова зависит от его окружения, например, в предложении “Он пошёл в ____, чтобы купить хлеб”, где для

предсказания слова «магазин» нужна информация как о начале, так и о конце предложения. Однако двунаправленное внимание неприменимо для автогенерации текста, как в GPT, где будущее не может быть учтено заранее.

1.1.2 Обучение BERT

Обучение BERT состоит из двух этапов: предобучение на неразмеченных данных и дообучение (fine-tuning) на размеченных данных под определенную задачу. В первом этапе модель учится на огромном количестве данных в течение длительного времени. Полученные веса возможно использовать как основу для решения других, более частных задач, для чего стоит всего лишь дообучить модель на небольшом объеме размеченных данных. Таким образом, второй этап представляет собой обычное обучение с учителем.

Для предобучения модели авторы BERT использовали две задачи обучения с самоконтролем: сначала используется маскированная языковая модель (masked language model), затем — предсказание следующего предложения.

1.1.3 Маскированная языковая модель с двунаправленным вниманием

Рассмотрим задачу MLM (Masked Language Modeling). Идея этой задачи проста: если у нас есть текстовые данные, давайте удалим часть слов и попробуем предсказать пропуски. Например, можно попробовать удалять последнее слово предложения и пытаться его предсказать. Однако при этом внимание модели будет сосредоточено только на контексте слева. Аналогично, если удалять первое слово, модель сосредоточится только на контексте справа. Чтобы учитывать оба направления контекста, попробуем восстанавливать пропуски слов в середине текста.

Интуитивно понятно, что модель с двунаправленным вниманием будет гораздо более мощная и более способная к обобщению, чем та, что использует классическое внимание (слева-направо) или конкатенацию направлений (слева-направо и справа-налево). Нам уже известно, как можно обучить модель с однонаправленным вниманием, но как обучить модель с двунаправленным? Вспомним, что при обучении модели с классическим вниманием мы используем Masked Multihead Attention. Это предотвращает утечку данных во время обучения, так как модель не может косвенно использовать целевое слово, опираясь на его присутствие в контексте. Однако для учета двунаправленного контекста использование Masked Multihead Attention становится невозможным, что требует другого подхода к обучению.

Для реализации задачи используется тренировочный генератор. Это функция, которая случайным образом выбирает определённый процент токенов из последовательности (в BERT это 15%) и модифицирует их следующим образом:

- в 80% случаев токен заменяется на специальный токен [MASK];

- в 10% — на случайный токен из словаря;
- в оставшихся 10% токен остаётся неизменным.

Во время обучения минимизируется кросс-энтропия между выходами модели - маскированными токенами, и соответствующими им немаскированными токенами.

1.1.4 Предсказание следующего предложения

Многие задачи обработки естественного языка (такие как Question Answering и Natural Language Inference) основаны на понимании отношений между двумя предложениями. Например, в задаче Question Answering модель должна найти ответ на заданный вопрос, анализируя текстовый контекст. В задаче Natural Language Inference требуется определить, являются ли два предложения взаимно исключающимися, следуют ли одно из другого или не связаны вовсе. Чтобы сделать BERT более универсальной моделью, авторы предобучают модель на задаче предсказания следующего предложения.

Для предобучения модели на такой задаче необходим некоторый корпус текста. Выберем предложения A и B таким образом, что в половине случаев B является продолжением предложения A (*IsNext*), а в другой половине — случайным предложением (*NotNext*). Чтобы модель могла разделять предложения, используется специальный токен [SEP], а для классификации задач *IsNext* и *NotNext* — токен [CLS].

1.2 GPT

Через год после изобретения трансформеров, в 2018-ом, исследователи OpenAI опубликовали статью под названием “Improving Language Understanding by Generative Pre-Training“, в которой они представили новую модель — GPT-1.

Несмотря на свой относительно небольшой размер (всего 117 миллионов параметров), GPT-1 продемонстрировала высокую производительность в нескольких задачах NLP. Эта модель проложила путь к созданию более мощных моделей с огромными наборами данных и многомиллиардным количеством параметров.

2 Мультимодальность

Мультимодальность в нейронных сетях означает способность модели обрабатывать информацию из нескольких типов данных — модальностей. Это могут быть изображения, видео, текст, аудио, а также другие формы данных.

Однако граница того, что мы можем называть модальностью — довольно зыбкая. Различные типы данных по-разному отличаются между собой, поэтому правильно будет сказать о переходе от более однородных и схожих качественно и структурно данных, к более разнородным. Например:

- Два изображения мелких камней на конвейерной ленте, снятые на одном заводе в один и тот же день, будут очень близки и, возможно, почти неотличимы. Это точно объекты одной модальности.
- Текст на естественном языке и программный код на высокоуровневом языке программирования будут различаться в своей структуре, синтаксисе и внутренней логике, но кодируются последовательностью символов, например, utf-8. Одну и ту же модель можно обучать как и на данных естественного языка, так и на данных кода. Это, скорее данные разных доменов, чем разных модальностей.
- Текст и изображения отличаются сильно и для их обработки требуются разные архитектуры. Это точно разные модальности.

2.1 Зачем нужна мультимодальность?

Однако, зачем моделям нужна мультимодальность? Поскольку нейронные сети и языковые модели в частности создаются для помощи людям, для решения многих задач могут потребоваться навыки в работе как с текстом, так и с изображениями. Например, пользователю необходимо найти в многостраничном справочнике трав и растений все изображения цветов “кошачья лапка”. Для определения цветка точно потребуется способность модели работать с изображениями. А точность ответа может быть повышена с помощью обработки текста энциклопедии.

Также, для обучения особо крупных моделей, необходимы особо крупные наборы данных. Использование различных модальностей позволяет прибегать к новым источникам информации при обучении моделей, что позволяет насытить датасеты свежими данными, полученными из модальностей изображений, аудио или, например, видео.

Теперь рассмотрим задачу генерации для GPT-подобного трансформера. Такая модель на этапе инференса должна генерировать последовательность новых токенов. Эти токены потом декодируются в текст. Но что, если модель была обучена на последовательности токенов как текста, так и изображений, аудио или видео? Тогда такая модель будет способна генерировать токены соответствующих модальностей, которые потом возможно декодировать подходящими детокенизаторами.

Полная реализация возможностей мультимодальных моделей открывает дорогу к созданию систем, способных в реальном времени анализировать видеоряд с экрана монитора, веб-камеры, способных “слушать” пользователя и одновременно генерировать ответ, как в виде текста, так и озвученный синтезированным голосом с соблюдением всех интонации, темпа речи и пауз.

Однако, мы рассмотрим мультимодальность только между двумя типами данных — текстом и изображениями.

2.2 Задача описания изображения

Одной из базовых задач мультимодальных моделей является задача генерации текстового описания изображения (image-captioning, image2text). Эта задача находится на стыке компьютерного зрения и обработки естественного языка, и очевидно потребует применения идей из обеих областей.

Нам уже известны разные кодировщики и декодировщики, как для задач компьютерного зрения, так и для задач обработки естественного языка. Например, ViT является трансформером-кодировщиком для обработки изображений, а GPT-2 является декодировщиком для задачи обработки текста. Наивное решение задачи — объединить два трансформера в модель кодировщик-декодировщик (наподобие оригинального трансформера). Поскольку требуемый выход модели — сгенерированный текст, то “основной” частью будет декодировщик, а информация об изображении будет “подмешиваться” в механизм внимания с помощью перекрестного внимания. Однако, в оригинальной архитектуре GPT-2 нет слоёв перекрестного внимания.

Исследователи Google Research в статье *Leveraging Pre-Trained Checkpoints for Sequence Generation Tasks* продемонстрировали, что можно инициализировать слои перекрестного внимания в GPT случайным образом без существенных потерь в качестве.

А теперь вспомним механизм внимания в трансформерах. Элементы матрицы внимания отображают близость векторов q_i и k_j — чем меньше угол между этими векторами, тем они больше значение соответствующего элемента матрицы внимания и тем больше “связь” между соответствующими токенами.

В перекрестном внимании матрица K — это матрица, которая приходит извне. В нашем случае это матрица, получаемая из последовательности токенов, полученной после проекции выхода ViT:

$$K = X_{\text{ViT}}W_K$$

А матрица Q — матрица, которая получается из последовательности токенов уже сгенерированного текста с помощью GPT-2:

$$Q = X_{\text{GPT}}W_Q$$

Мы не можем гарантировать, что X_{ViT} и X_{GPT} лежат в одном семантическом пространстве. Конечно, проекции W_Q и W_V могут все исправить, однако это дополнительно усложнит решаемую задачу. Можем ли мы заранее обучить ViT так, чтобы его выходы уже лежали в том же семантическом пространстве, что и токены текста?

2.3 CLIP

В 2021ом году исследователи OpenAI в статье *Learning Transferable Visual Models From Natural Language Supervision* предложили свое решение — модель, состоящую из text encoder и image encoder, — Contrastive Language-Image Pre-training (CLIP).

CLIP очень хорошо справляется с задачей определения соответствия изображения и текста. Это свойство востребовано во множестве других задач помимо text2image генерации. В оригинальной статье авторы даже смогли таким образом классифицировать изображения из датасета ImageNet. CLIP способен к обобщению информации и распознаванию образов без явного дообучения, и, что впечатляет, может распознавать геолокацию, различные действия и так далее. Задача распознавания текста на изображении однако не дается CLIP’у.

Способности CLIP к zero-shot распознаванию образов (то есть способность давать осмысленные ответы на задачи без примеров) впечатляющи, но не неожиданны. Аналогичное происходило с GPT-3, обученном на огромном массиве данных. CLIP, в свою очередь, был обучен на 400 миллионах пар “изображение- текст”.

Обучение CLIP происходит следующим образом:

- возьмем батч изображений и батч текстов;
- изображения обрабатываются кодировщиком изображений (в данном случае ViT), текст - кодировщиком текста (авторы используют свою архитектуру);
- для изображений получается набор векторов $\{I_k\}_{k=1}^{\text{batch_size}}$; для текстов - $\{\tau_k\}_{k=1}^{\text{batch_size}}$;
- далее происходит самый важный шаг - вычисление косинусной близости между парами $\{I_k, \tau_j\}$.

$$\text{cosine_similarity}(I_k, \tau_j) = \frac{(I_k, \tau_j)}{\|I_k\| \cdot \|\tau_j\|},$$

Для пар $\{I_k, \tau_k\}$ (с одинаковым индексом) косинусная близость максимизируется, а для всех остальных случаев - минимизируется. В оригинальной статье также приводится пример дообучения на классификацию изображений из датасета ImageNet.

2.4 BLIP

CLIP был обучен на 400 миллионах пар текст-изображение. Это много. Причём, данный текст так или иначе требует ручной разметки: очистки от некорректных объектов, исправление и тд. Без этого этапа, качество модели значительно снизится.

В статье **BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation** предложена модель BLIP. Её идея заключается в улучшении качества обучающих данных с помощью самой модели.

На первом этапе обучается модель, генерирующая подписи к изображениям. Далее используется модель-фильтр, отсеивающая некачественные

подписи. Этот подход позволяет модели как минимизировать влияние некачественных данных, так и обогатить их сгенерированными описаниями, благодаря чему основная модель обучается эффективнее.

2.5 Кросс-модальность трансформеров

Как видно, трансформеры способны успешно обрабатывать и текст, и изображения. Почему так, ведь эти две модальности отличаются гораздо сильнее, чем, например, текст от временных рядов или изображения от мел-спектрограмм (используемых для работы с аудио). Что же на самом деле делает текст и изображения различными, и как это отражается в моделях трансформеров?

Во-первых, текст — это последовательность. В отличие от LSTM, где последовательное восприятие информации заложено в модели архитектурно, трансформер воспринимает текст как множество, рассматривая токены параллельно. В свою очередь, изображения хоть и считываются в виде матрицы, в трансформере представляются как последовательность векторов-участков этой самой матрицы. Этот подход позволяет обрабатывать изображение похожим с текстом образом — в виде множества. Такое множество возможно объединить с множеством текстовых векторов и обрабатывать их вместе.

Теперь поговорим о проблеме с обработкой изображений трансформерами. Вспомним, что текст — семантически плотная структура. Удаление даже одного слова способно в корне изменить смысл предложения. Изображения же семантически разреженные и их “смысловое наполнение”, в основном, формируется низкочастотными структурами. Благодаря этому удаление одного пикселя или небольшой группы пикселей может быть даже не заметно. При проектировке мультимодальных моделей стоит учитывать эту разницу при объединении текстовых эмбеддингов и эмбеддингов изображения.

2.6 Смешивание модальностей

Помимо того, как представлять данные разных модальностей в моделях, следует задуматься о том, как эти представления объединять. Самый эффективный подход - кодирование признаков различных модальностей в латентное пространство и генерация результата с использованием предобученной модели. Рассмотрим варианты объединения нескольких модальностей.

2.6.1 Early Fusion

Этот подход смешивания модальностей подразумевает объединение выходов кодировщика каждой модальности и подачи результата в модель генерации (основную-модель), это может быть например LLM. Токены не обязательно будут лежать даже в пространстве одной размерности. Для проецирования признаков в единое пространство, используется **сеть-адаптер**.

Такой способ позволяет значительно экономить данные и ресурсы при дообучении, ведь все параметры модели, кроме параметров сетей-адаптеров, заморожены.

2.6.2 Deep Fusion

При таком подходе смешивания на вход модели подается информация с “главной” модальности, а информация о других модальностях поступает параллельно. Что-то подобное мы реализовывали для наивного решения задачи генерации описания изображения. Для “вспомогательных” модальностей можно также использовать свои адаптеры. При смешивании модальностей отличным способом передачи информации в основную модель является использование механизма cross-attention.

Deep Fusion позволяет сильнее всего учитывать связи между модальностями внутри модели, однако является сложным в реализации и требует значительно больших вычислительных ресурсов при дообучении, чем Early Fusion. Такой подход может быть использован при обучении больших мультимодальных моделей с нуля.

2.6.3 Late Fusion

Late Fusion предполагает объединение данных на самом последнем этапе, перед финальным выходом модели. На примере задачи классификации - это может быть объединение логитов.

Late Fusion - простой и дешевый способ в реализации. На практике такой подход почти не дает прироста в качестве модели, потому что при таком подходе почти не учитываются связи между двумя модальностями.

2.6.4 Merge of Encoders

Обычно для обработки каждой модальности используется один кодировщик. Такие мультимодальные модели умеют решать общие задачи, например, генерацию описания. Предположим, мы хотим сделать модель для анализа финансовых отчетов в компании. Тогда она должна понимать диаграммы, таблицы финансов, распознавать текст и при этом иметь возможность эти отчеты исправлять - указывать на ошибки в диаграммах или показывать недочеты в таблицах. Очевидно, дообучать такую модель стоит очень дорого по вычислительным и временным ресурсам.

Мы можем подойти к проблеме с другой стороны: возьмем несколько обученных под частные задачи кодировщиков и их выходы объединим в эмбединги для модели генерации. Таким образом наша модель получит возможность решать эту частную задачу. Можно сравнить это с конструктором: вы купили себе новый набор и теперь способны творить намного больше.

Рассмотрим способы объединения эмбедингов:

- **Pixel-shuffle.** Увеличение ширины эмбедингов за счет уменьшения длины последовательности через реорганизацию векторов. Последующее приведение эмбедингов к необходимой размерности возможно при помощи одного общего адаптера (вместо отдельных адаптеров для каждой модели).
- **Аддитивное смешивание.** Если размерности выходов кодировщиков совпадают, то итоговый вектор можно представить как взвешенную сумму между векторами разных кодировщиков

$$z = \sum_{i=1}^k w_i x_i,$$

где веса w_i - обучаемы.

- **Мультипликативное смешивание.** Аналогично аддитивному смешиванию, возможно произведение векторов модальностей с умножением на некоторый параметр w :

$$z = w(x_1 \times x_2)$$

- **Gated Fusion.** Данный подход похож на аддитивное смешивание, однако вместо параметров w_i используются функции внимания g_i :

$$z = \sum_{i=1}^k g_i(x_1, x_2, \dots, x_k) x_i$$

3 Законы масштабирования

3.1 Масштабирование нейронных сетей

Когда мы обучаем модель размером N на наборе данных объёма D , функция потерь $\mathcal{L}$ на тестовой выборке сначала снижается быстро. Однако чем больше проходит шагов обучения, тем медленнее она убывает. При условии, что модель не переобучается, процесс тренировки может продолжаться довольно долго, и чем дольше мы обучаем модель, тем ниже будет $\mathcal{L}$ на тестовом датасете.

Очевидно, что повысить качество модели можно за счёт увеличения N и D . Увеличение числа параметров, размера датасета и количества вычислений C (compute) называется масштабированием нейронных сетей.

Архитектура трансформера позволяет гибко увеличивать количество параметров нейронной сети. Использование одного или множества GPU и применение различных стратегий распределённого обучения дают возможность масштабировать вычисления. А self-supervised обучение в теории

открывает доступ к данным всего интернета. Однако, обучение многомиллиардных моделей с привлечением огромного объёма вычислений — занятие крайне дорогостоящее, поэтому выходит, что масштабирование ограничено количеством вычислений.

В работах *Scaling Laws for Autoregressive Generative Modelling* и *Scaling Laws for Language Models* (OpenAI) были продемонстрированы законы масштабирования — зависимости $\mathcal{L}$ на тестовой выборке от объёма вычислений C , размера модели N и размера датасета D .

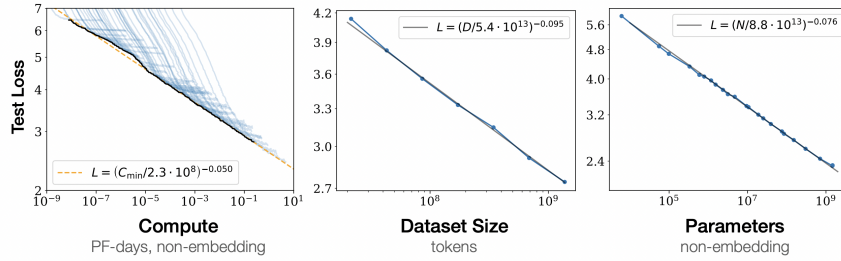


Рис. 1: Законы масштабирования для задачи языкового моделирования.

Для задачи языкового моделирования оказалось, что лосс на логарифмическом графике сходится к следующим прямым:

$$\mathcal{L} \sim \left(\frac{1}{7.3} C \cdot 10^8 \right)^{-0.050}, \quad \mathcal{L} \sim \left(\frac{1}{5.4} D \cdot 10^{11} \right)^{-0.095}, \quad \mathcal{L} \sim \left(\frac{1}{8.8} N \cdot 10^{13} \right)^{-0.027}$$

Однако достичь нулевого значения функции потерь невозможно из-за энтропии данных. Для текстовых данных энтропия выражается в неопределённости предсказания следующего токена. Например, модель, обученная на всём интернете, не сможет однозначно (с вероятностью 1.0) предсказать следующий токен для фразы “искусственный интеллект — это“, поскольку существует множество разных определений искусственного интеллекта. Таким образом, как бы мы ни увеличивали N , D или C , опустить $\mathcal{L}$ ниже значения энтропии данных не получится.

Так, изучая законы масштабирования для других задач, исследователи OpenAI заметили, что кривые трендов сглаживаются. Оказалось, что энтропия данных, вычисленная при масштабировании размера модели, а также энтропия, рассчитанная при масштабировании количества вычислений, практически совпадают. Однако, специалисты OpenAI не смогли оценить энтропию текстовых данных, что правда не означает того, что она нулевая.

Так, законы масштабирования стали теоретической основой появления больших языковых моделей и заложили курс OpenAI на увеличение размера моделей и количества вычислений.

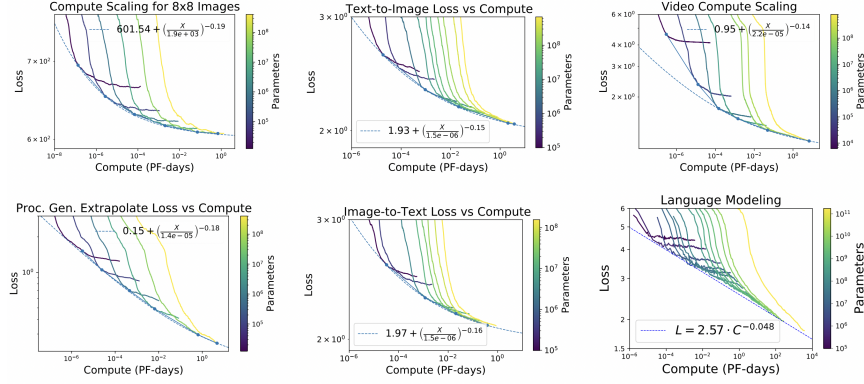


Рис. 2: Законы масштабирования для различных доменов

3.2 Законы масштабирования и bias-variance tradeoff

Запишем законы масштабирования для количества параметров и количества данных:

$$\mathcal{L}(N) = c + aN^{-\alpha}, \quad \mathcal{L}(D) = c + bD^{-\beta}$$

Совместный закон масштабирования для N и D одновременно, полученный эмпирически, можно записать следующим образом:

$$\begin{cases} \mathcal{L}(N, D) = c + AN^{-\alpha} + BD^{-\beta}, \\ C(N, D) = C_0 ND \end{cases}$$

Этот закон масштабирования повторяет разложение достижимой функции потерь на смещение (bias) и разброс (variance), где

$$\text{bias} \sim N^{-\alpha}, \quad \text{variance} \sim D^{-\beta}$$

Из совместного закона масштабирования можно прийти к ранее полученному выводу: увеличение размера модели снижает смещение, увеличение количества данных снижает разброс. Рассмотрим соотношение:

$$R(N, D) = \frac{\text{variance}}{\text{bias}} \sim N^{\alpha} D^{-\beta}$$

При $R(N, D) \ll 1$, то есть если модель недообучена, получается:

$$\mathcal{L}(N, D) \approx c + AN^{-\alpha}$$

При $R^{-1}(N, D) \ll 1$, то есть если модель переобучена, получается:

$$\mathcal{L}(N, D) \approx c + BD^{-\beta}$$

Поскольку количество вычислений фиксировано $C \sim ND$, то для получения минимального $\mathcal{L}(N, D)$ необходимо найти кривую $D = f(N)$, гарантирующую оптимальное соотношение $R(N, D)$.

3.3 Закон Шиншиллы

Вернемся к закону $\mathcal{L}(N, D)$:

$$\begin{cases} \mathcal{L}(N, D) = c + AN^{-\alpha} + BD^{-\beta}, \\ C(N, D) = C_0 ND \end{cases}$$

Команда DeepMind в статье *Training Compute-Optimal Large Language Models* поставила задачу найти выражение $N = f(D)$, чтобы получить наилучшую модель при фиксированном бюджете вычислений $C \sim ND$. Для совместного закона масштабирования они эмпирически нашли следующие значения:

$$\alpha = 0.34, \quad \beta = 0.28, \\ A = 406.4, \quad B = 410.7, \quad L_0 = 1.82, \quad C_0 = 6.$$

Авторы применили несколько методов для нахождения оптимального соотношения количества параметров и объёма данных. Рассмотрим два из них.

Первый метод заключается в нахождении минимума по объединению кривых $\mathcal{L}(C)$ для разных моделей размером от 10М до 70В на датасетах от 5В до 500В токенов.

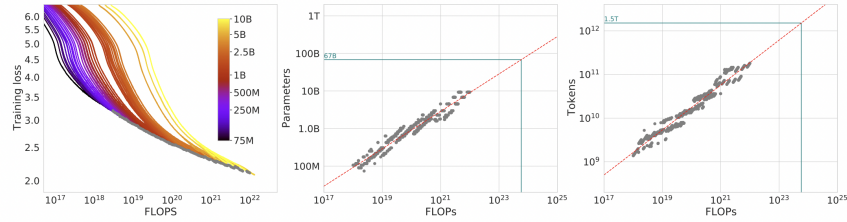


Рис. 3: Минимум объединения кривых

Второй метод основывается на анализе IsoFLOPs кривых. IsoFLOPs кривые — это линии на графике $\mathcal{L}(N)$ при $C = \text{const}$. В ходе экспериментов получилось семейство кривых похожей формы и с одним минимумом каждая. Тренд этих минимумов для $N(C)$ и $D(C)$ оказался прямой линией. Иными словами, это выражение $D = f(N)$, гарантирующее оптимальный bias-variance trade-off.

Таким образом, в обоих случаях было получено примерно одинаковое оптимальное соотношение количества параметров и количества токенов:

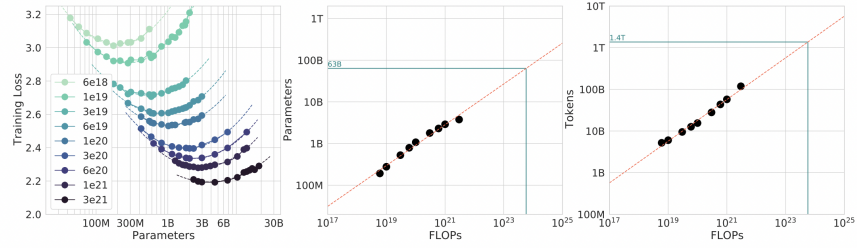


Рис. 4: Анализ IsoFLOPs

$$D = 20N$$

Это значительно большее соотношение, чем было использовано при обучении моделей GPT-3 175B ($D \approx 1.7N$), Gopher 280B ($D \approx 1.1N$), Jurassic-1 178B ($D \approx 1.7N$), Megatron 530B ($D \approx 0.5N$). низкое соотношение количества токенов к количеству параметров приводит к высокому значению variance.

В доказательство корректности своего вывода, исследователи DeepMind обучили модель Chinchilla 70B при $D = 20N$ и при бюджете Gopher. Такая модель показала значительно лучшие результаты по сравнению с перечисленными выше.

4 МоЕ